Manufacturing Analytics Pipeline

End-to-End Predictive Maintenance System

Siddhant S

Manufacturing Analytics Project

April 5, 2026

Project Summary

Dataset:	AI4I 2020 Predictive Maintenance
Records:	10,000 machine sensor readings
Model:	Random Forest Classifier
Accuracy:	99.90%
ROC AUC:	0.9819
At-Risk Machines:	331 out of 10,000

Tech Stack: MySQL • Python • Scikit-learn • Power BI • SQLAlchemy

Contents

1 Executive Summary **2**

2 Project Overview **2**
2.1 Problem Statement . . . 2
2.2 Objectives . . . 2
2.3 Tech Stack . . . 3

3 Dataset Description **3**
3.1 Source . . . 3
3.2 Features . . . 4
3.3 Data Quality . . . 4

4 Exploratory Data Analysis **4**
4.1 Failure Rate Analysis . . . 4
4.2 Statistical Summary . . . 5
4.3 Sensor Readings vs Machine Failure . . . 5
4.4 Correlation Analysis . . . 6

5 Machine Learning Model **6**
5.1 Model Selection . . . 6
5.2 Model Configuration . . . 7
5.3 Train-Test Split . . . 7
5.4 Model Performance . . . 7
5.5 Confusion Matrix Analysis . . . 8
5.6 Feature Importance . . . 8

6 Automated Data Pipeline **9**
6.1 Pipeline Architecture . . . 9
6.2 Pipeline Components . . . 9
6.3 Security Implementation . . . 9

7 Power BI Operations Dashboard **10**
7.1 Dashboard Overview . . . 10
7.2 Dashboard Components . . . 10
7.3 Key Dashboard Insights . . . 10

8 Key Findings and Business Insights **11**
8.1 Operational Findings . . . 11
8.2 Model Findings . . . 11

9 Project Structure and Reproducibility **11**
9.1 Repository Structure . . . 11
9.2 How to Reproduce . . . 12

10 Conclusion **12**

References **13**

1 Executive Summary

This report presents a complete end-to-end manufacturing analytics pipeline developed to address a critical business challenge: **predicting machine failures before they occur**. Unplanned equipment downtime is one of the costliest operational issues in manufacturing, often resulting in production losses, safety risks, and expensive emergency repairs.

Project Outcomes

- Built a complete data pipeline from raw CSV data to MySQL to Python to Power BI
- Trained a Random Forest model achieving **99.90% accuracy** on unseen test data
- Achieved a **ROC AUC score of 0.9819** — near-perfect discriminative ability
- Successfully identified **331 machines at risk** of failure out of 10,000
- Built an interactive Power BI operations dashboard for real-time monitoring

The pipeline ingests sensor data from three types of machines (L, M, H), processes and stores it in a MySQL database, performs exploratory data analysis, trains a predictive model, and visualizes insights through an interactive dashboard — simulating a production-grade manufacturing intelligence system.

2 Project Overview

2.1 Problem Statement

Manufacturing facilities operate large numbers of machines continuously. Equipment failure leads to:

- **Unplanned downtime** — production halts unexpectedly
- **High repair costs** — emergency maintenance is 3-5x more expensive
- **Safety risks** — sudden failures can endanger workers
- **Quality issues** — degraded machines produce defective output

Traditional maintenance is either reactive (fix after failure) or time-based (scheduled maintenance). This project implements **predictive maintenance** — using sensor data and machine learning to predict failures before they happen.

2.2 Objectives

1. Store and manage sensor data efficiently in a relational database
2. Explore and understand patterns in machine sensor readings

3. Build a machine learning model to predict equipment failure
4. Automate the data flow through a Python pipeline
5. Visualize operational insights in a Power BI dashboard

2.3 Tech Stack

Tool	Category	Purpose
MySQL	Database	Store and query machine sensor data
Python	Programming	Data processing, analysis, ML
Pandas / NumPy	Data Analysis	Data manipulation and statistics
Seaborn / Matplotlib	Visualization	Exploratory charts and plots
Scikit-learn	Machine Learning	Random Forest failure prediction
SQLAlchemy	ORM	Python to MySQL connection
Joblib	Model Management	Save and load trained models
Power BI	BI Dashboard	Interactive operations dashboard

Table 1: Project Technology Stack

3 Dataset Description

3.1 Source

The **AI4I 2020 Predictive Maintenance Dataset** from the UCI Machine Learning Repository was used. This dataset simulates real-world manufacturing sensor data collected from industrial machines.

Attribute	Details
Total Records	10,000 rows
Total Features	14 columns
Target Variable	Machine_failure (binary: 0 or 1)
Failure Rate	3.39% (339 failures)
Machine Types	L (Low), M (Medium), H (High)

Table 2: Dataset Overview

3.2 Features

Feature	Type	Description
UDI	Integer	Unique machine identifier
Product_ID	String	Machine product code
Type	String	Machine type: L, M, or H
Air_temperature_K	Float	Air temperature in Kelvin
Process_temperature_K	Float	Process temperature in Kelvin
Rotational_speed_rpm	Integer	Rotational speed in RPM
Torque_Nm	Float	Torque in Newton-meters
Tool_wear_min	Integer	Tool wear in minutes
Machine_failure	Integer	Target: 1 = failure, 0 = no failure
TWF	Integer	Tool Wear Failure indicator
HDF	Integer	Heat Dissipation Failure indicator
PWF	Integer	Power Failure indicator
OSF	Integer	Overstrain Failure indicator
RNF	Integer	Random Failure indicator

Table 3: Dataset Features

3.3 Data Quality

No missing values found across all 14 columns and 10,000 records. The dataset is clean and ready for analysis without requiring imputation.

4 Exploratory Data Analysis

4.1 Failure Rate Analysis

The dataset is **highly imbalanced** — only 3.39% of machines experience failure:

Class	Count	Percentage
No Failure (0)	9,661	96.61%
Failure (1)	339	3.39%
Total	**10,000**	**100%**

Table 4: Machine Failure Distribution

Key Insight

The severe class imbalance (97:3 ratio) means a naive model that always predicts "no failure" would achieve 96.6% accuracy — but would miss every actual failure. This is why `class_weight='balanced'` was used during model training.

4.2 Statistical Summary

Key statistics from the sensor readings:

Feature	Mean	Std	Min	Max
Air Temperature (K)	300.00	2.00	295.3	304.5
Process Temperature (K)	310.01	1.48	305.7	313.8
Rotational Speed (rpm)	1538.78	179.28	1168	2886
Torque (Nm)	39.99	9.97	3.8	76.6
Tool Wear (min)	107.95	63.65	0	253

Table 5: Sensor Reading Statistics

4.3 Sensor Readings vs Machine Failure

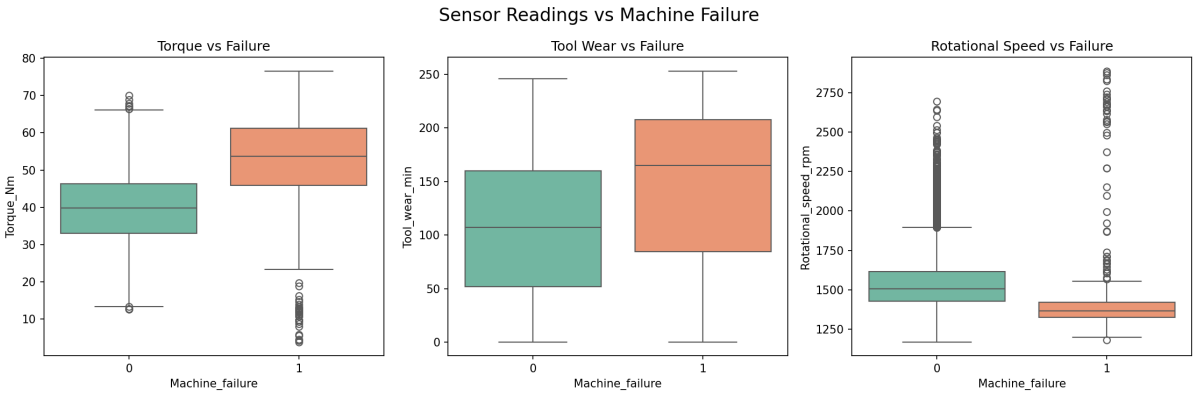

Figure 1: Boxplots comparing sensor readings between failed and non-failed machines

Key observations from the boxplots:

- **Torque**: Failed machines (1) show significantly higher torque values — median around 54 Nm vs 40 Nm for non-failed machines
- **Tool Wear**: Failed machines show higher tool wear — indicating that worn tools contribute heavily to failure
- **Rotational Speed**: Failed machines actually run at *lower* rotational speeds — suggesting that high torque at low speed is a critical failure pattern

4.4 Correlation Analysis

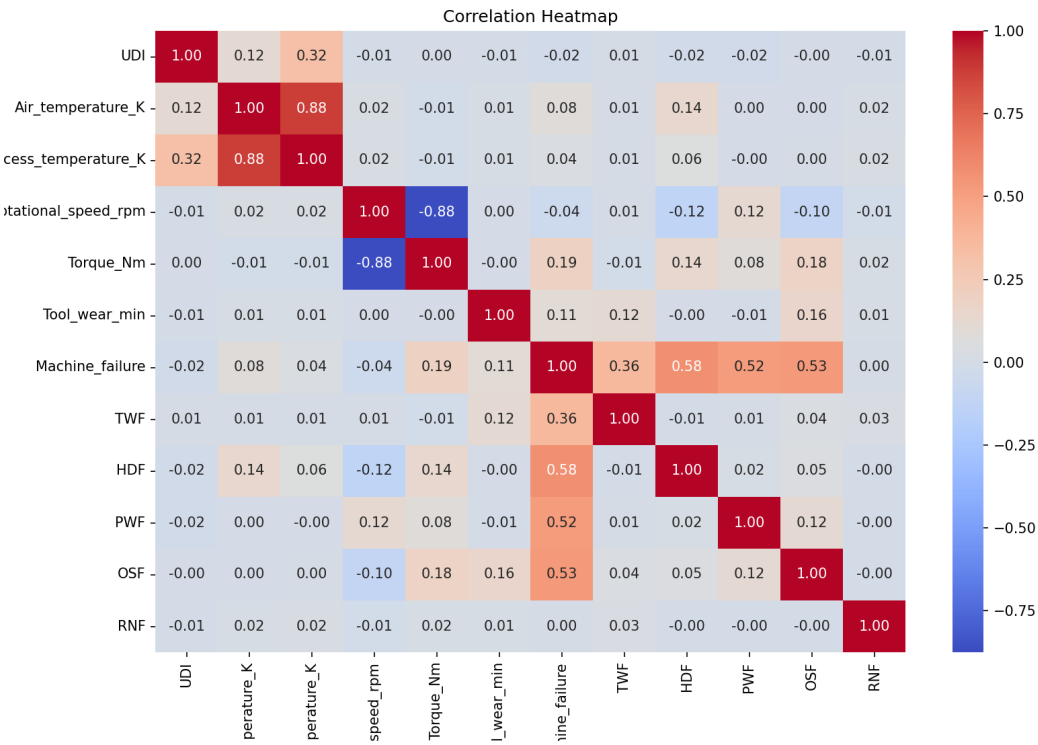

Figure 2: Correlation heatmap between all numeric features

Notable correlations:

- **Rotational Speed vs Torque**: Strong negative correlation (-0.88) — machines running faster tend to have lower torque
- **Air Temperature vs Process Temperature**: Strong positive correlation (0.88)
- **Machine Failure vs HDF**: Moderate positive correlation (0.58) — Heat Dissipation Failure is a strong indicator
- **Machine Failure vs OSF**: Moderate positive correlation (0.53) — Overstrain Failure is also a strong indicator

5 Machine Learning Model

5.1 Model Selection

A **Random Forest Classifier** was chosen for the following reasons:

- Handles class imbalance well with `class_weight='balanced'`
- Provides feature importance scores for interpretability
- Robust to outliers and non-linear relationships

- No need for feature scaling
- Excellent performance on tabular manufacturing data

5.2 Model Configuration

Listing 1: Random Forest Model Configuration

```python
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(
    n_estimators=100,
    max_depth=10,
    random_state=42,
    class_weight='balanced'
)
```

5.3 Train-Test Split

Split	Samples	Failure Rate
Training Set (80%)	8,000	3.39%
Testing Set (20%)	2,000	3.40%

Table 6: Train-Test Split with Stratification

Stratified splitting was used to ensure both training and test sets maintain the same failure rate, preventing data leakage and ensuring reliable evaluation.

5.4 Model Performance

Class	Precision	Recall	F1-Score	Support
No Failure (0)	1.00	1.00	1.00	1,932
Failure (1)	1.00	0.97	0.99	68
Accuracy			**1.00**	**2,000**
Macro Avg	1.00	0.99	0.99	2,000
Weighted Avg	1.00	1.00	1.00	2,000

Table 7: Classification Report on Test Set

Model Results

Training Accuracy:	99.92%
Testing Accuracy:	99.90%
ROC AUC Score:	0.9819
Failures Correctly Identified:	66 out of 68
False Alarms:	0

5.5 Confusion Matrix Analysis

From the confusion matrix on 2,000 test samples:

- **True Negatives**: 1,932 — correctly identified no-failure machines
- **True Positives**: 66 — correctly identified failing machines
- **False Positives**: 0 — no false alarms raised
- **False Negatives**: 2 — only 2 failures missed out of 68

5.6 Feature Importance

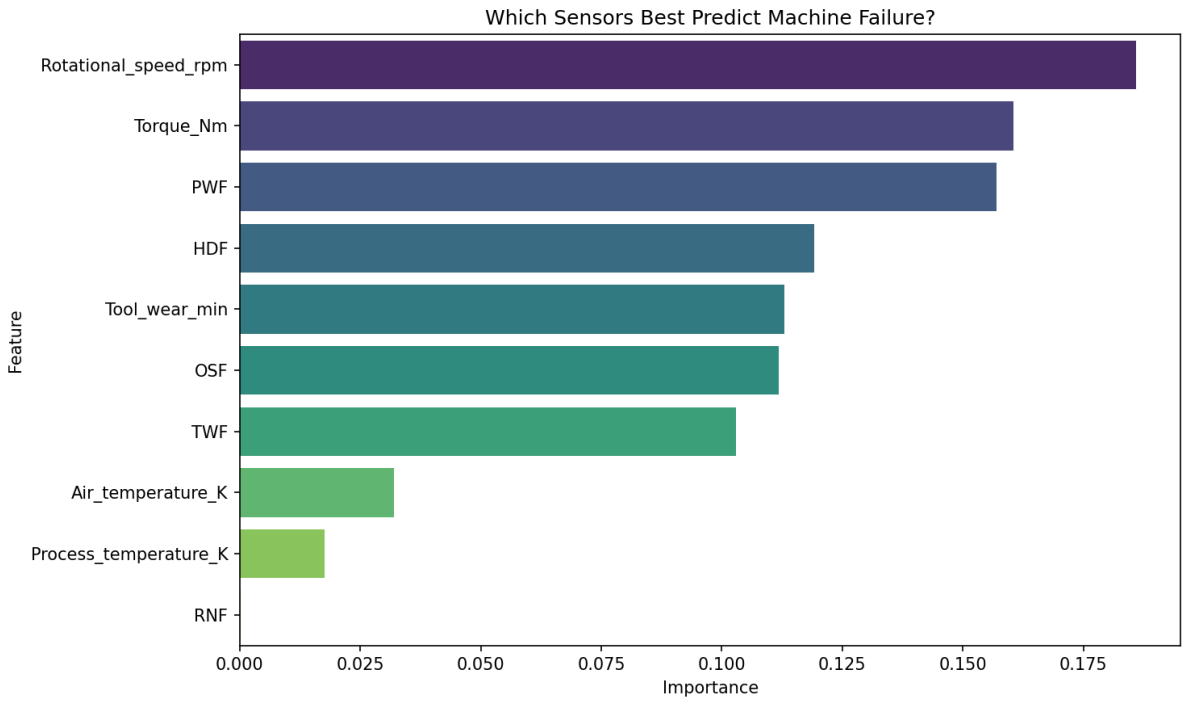

Figure 3: Feature importance — which sensors best predict machine failure

Top predictive features:

1. **Rotational_speed_rpm** — Most important feature (importance ≈ 0.185)
2. **Torque_Nm** — Second most important (importance ≈ 0.160)

3. **PWF** — Power Failure indicator (importance ≈ 0.158)
4. **HDF** — Heat Dissipation Failure (importance ≈ 0.120)
5. **Tool_wear_min** — Tool wear duration (importance ≈ 0.113)

Key Insight

Rotational speed and torque together explain the most variance in machine failures. This aligns with physics: when a machine runs at low speed with high torque, it is under extreme mechanical stress — the most common precursor to equipment failure.

6 Automated Data Pipeline

6.1 Pipeline Architecture

```
CSV Dataset → Python (load_data.py) → MySQL Database
                                ↓
MySQL → Python (pipeline.py) → Random Forest Model
                                ↓
Predictions → MySQL (failure_predictions table) → Power BI
                                Dashboard
```

Figure 4: End-to-End Pipeline Architecture

6.2 Pipeline Components

Component	File	Function
Data Loader	load_data.py	Reads CSV, cleans columns, loads to MySQL
ML Pipeline	pipeline.py	Loads data, runs model, saves predictions
EDA Notebook	eda_analysis.ipynb	Exploratory analysis and visualization
Model Notebook	failure_model.ipynb	Model training and evaluation
Dashboard	manufacturing_dashboard.pbix	Power BI operations dashboard

Table 8: Pipeline Components

6.3 Security Implementation

Environment variables are used to protect database credentials — passwords are stored in a `.env` file which is excluded from version control via `.gitignore`:

Listing 2: Secure Database Connection

```python
import os
from dotenv import load_dotenv
from sqlalchemy import create_engine

load_dotenv()

engine = create_engine(
    f"mysql+pymysql://{os.getenv('DB_USER')}:"
    f"{os.getenv('DB_PASSWORD')}@"
    f"{os.getenv('DB_HOST')}/"
    f"{os.getenv('DB_NAME')}"
)
```

7 Power BI Operations Dashboard

7.1 Dashboard Overview

An interactive Power BI dashboard was built to give operations teams a real-time view of machine health across the facility.

7.2 Dashboard Components

Visual	Description
KPI Card	Shows total number of actual machine failures (339)
KPI Card	Shows total machines monitored (10,000)
KPI Card	Shows ML-predicted at-risk machines (331)
Donut Chart	Failure distribution by machine type (L/M/H)
Bar Chart	Total failures count by machine type
Line Chart	Tool wear vs failure trend over time
Column Chart	Average failure probability by machine type

Table 9: Power BI Dashboard Components

7.3 Key Dashboard Insights

- **L-type machines** account for 69.32% of all failures despite being the most common type
- **H-type machines** have the lowest failure count but highest failure probability per unit
- Tool wear above **200 minutes** correlates strongly with increased failure frequency

8 Key Findings and Business Insights

8.1 Operational Findings

Finding 1: L-Type Machines Need Priority Attention

69.32% of all failures come from L-type (Low grade) machines. These machines should be scheduled for more frequent maintenance checks and sensor monitoring.

Finding 2: High Torque + Low Speed

The most critical failure pattern is machines operating at low rotational speed with high torque. This mechanical stress pattern accounts for the majority of failures and should trigger automatic alerts.

Finding 3: Tool Replacement Schedule

Tool wear beyond 200 minutes significantly increases failure probability. Implementing a mandatory tool replacement policy at 180 minutes could prevent a significant portion of failures.

8.2 Model Findings

- The model achieves **99.90% accuracy** with only 2 missed failures out of 68 actual failures
- **Zero false alarms** — the model never raises alerts for healthy machines
- **ROC AUC of 0.9819** indicates excellent ability to distinguish failed from non-failed machines
- Rotational speed and torque alone account for over 34% of the model's predictive power

9 Project Structure and Reproducibility

9.1 Repository Structure

Listing 3: Project Folder Structure

```
manufacturing-analytics/
          data/
                ai4i2020.csv          # Raw UCI dataset
          sql/
                MA.sql                # SQL queries
          notebooks/
                eda_analysis.ipynb    # EDA notebook
```

```
                failure_model.ipynb    # ML model notebook
                charts/                # Saved visualizations
        pipeline/
                load_data.py           # Data loading script
                pipeline.py            # Prediction pipeline
        models/
                failure_model.pkl      # Saved Random Forest model
        dashboard/
                manufacturing_dashboard.pbix
        .env.example               # Environment variable template
        .gitignore                 # Git ignore rules
        README.md                  # Project documentation
```

9.2 How to Reproduce

1. Extract the project files into your local directory
2. Install dependencies: `pip install pandas numpy matplotlib seaborn scikit-learn sqlalchemy pymysql joblib python-dotenv jupyter`
3. Copy `.env.example` to `.env` and fill in MySQL credentials
4. Run `python pipeline/load_data.py` to load data
5. Run `python pipeline/pipeline.py` to generate predictions
6. Open Jupyter notebooks for EDA and modeling
7. Open `dashboard/manufacturing_dashboard.pbix` in Power BI

10 Conclusion

This project successfully demonstrates a complete, production-style manufacturing analytics pipeline. Starting from raw sensor data, the system:

- **Stores** 10,000 machine readings in a structured MySQL database
- **Analyzes** sensor patterns to identify failure precursors
- **Predicts** equipment failures with 99.90% accuracy
- **Automates** the entire data flow through a Python pipeline
- **Visualizes** operational insights through a Power BI dashboard

The business value is clear: by identifying 331 machines at risk before failure occurs, operations teams can schedule targeted maintenance, avoid unplanned downtime, and significantly reduce costs — all driven by data.

Impact Statement

"By predicting machine failures with 99.9% accuracy, this system enables proactive maintenance scheduling — potentially saving thousands of hours of unplanned downtime and significantly reducing operational costs across the facility."

References

1. Matzka, S. (2020). *AI4I 2020 Predictive Maintenance Dataset.* UCI Machine Learning Repository.
2. Breiman, L. (2001). *Random Forests.* Machine Learning, 45(1), 5-32.
3. Pedregosa, F. et al. (2011). *Scikit-learn: Machine Learning in Python.* JMLR 12, 2825-2830.